Lehrstuhl für Angewandte Mechanik
TUM School of Engineering and Design
Technische Universität München

Roboterdynamik – Praktikum (Modul MW2339)

Inverse Kinematik

In diesem Teil des Praktikums soll die Inverse Kinematik für Industrieroboter mit Drehgelenken implementiert werden. Die grundlegende Struktur ist wieder als Python-Code vorhanden.

1 Inverse Kinematik auf Geschwindigkeitsebene

Gegeben ist eine Trajektorie im Arbeitsraum, die in den Gelenkraum überführt werden soll. Wir nehmen an, dass die erste Position der Arbeitsraumtrajektorie mit der aktuellen Position des Roboters übereinstimmt. Somit kennen wir auch die aktuelle Gelenkwinkelkonfiguration des Roboters.

1.1 Resolved Motion Rate Control

In der Inversen Kinematik soll die Position des TCP und die Orientierung des letzten Körpers berücksichtigt werden. Vervollständigen Sie die Funktion calculate_ik_rmc der Klasse IKSolverVelocity in der Datei rdplib / ik .py.

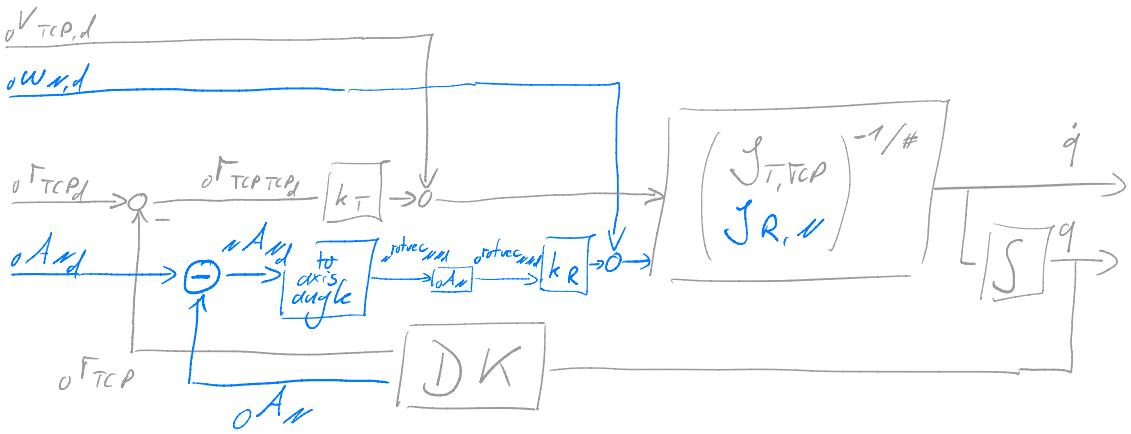


Abbildung 1: IK mit Position und Orientierung

Im Blockschaltbild Abb. 1 wird die kombinierte Jacobi-Matrix

$$\boldsymbol{J} = \begin{pmatrix} \boldsymbol{J}_{T,\mathrm{TCP}} \\ \boldsymbol{J}_{R,N} \end{pmatrix}$$

verwendet. Mit $\boldsymbol{J}^{\#}$ werden die translatorische Geschwindigkeit $\boldsymbol{v}_{\mathrm{TCP}}$ und die rotatorische Geschwindigkeit $\boldsymbol{\omega}_{\mathrm{TCP}}$ in den Gelenkraum übersetzt. Bei der Driftkompensation der Orientierung muss ein bisschen aufgepasst werden.

- Den Unterschied zwischen zwei Drehmatrizen erhält man aus einem Produkt, keiner Differenz.
- Die Drehmatrix ${}_N\boldsymbol{A}_{N_d}$ nutzen wir normalerweise, um Größen dargestellt in der Basis N_d in die Basis N zu transformieren. Drehmatrizen können auch verwendet werden, um Körper zu drehen – und das ist, was wir hier machen möchten! Die Drehmatrix ${}_N\boldsymbol{A}_{N_d}$ dreht uns einen Körper von N nach N_d.
- Diese kleine Differenzorientierung muss dann von der Drehmatrixdarstellung in die Axis-Angle-Darstellung übersetzt werden. Verwenden Sie hierfür scipy. spatial .transform.Rotation.from_matrix um ein Rotationsobjekt zu erzeugen. Verwenden Sie dann scipy. spatial .transform.Rotation.as_rotvec um die Axis-Angle-Darstellung zu erhalten.

- Der Rotatonsvektor (Axis-Angle-Darstellung) ist jetzt auch in einer Basis dargestellt! Beachten Sie dies, wenn Sie diesen Vektor mit der Winkelgeschwindigkeit addieren!

$$ {}_{N_d}\boldsymbol{A}_{N\ N}\boldsymbol{rotvec}_{N\,N_d} = {}_{N_d}\boldsymbol{rotvec}_{N\,N_d} $$

Beginnen Sie zunächst nur die IK für die Position des TCP zu implementieren. Betrachten Sie das Ergebnis im Web-Viewer. Ergänzen Sie dann die fehlenden Aspekte für die Orientierung.

1.2 Automatic Supervisory Control

Wir wechseln jetzt zum 7-achsigen Franka Roboter um trotz gewöhnlicher Arbeitsraumbeschreibung mit 6 Freiheitsgraden redundant zu sein. Auch für den Franka Robot bieten wir einen Viewer an. Kommentieren Sie die passenden Zeilen in inverse_kinematics_trajectory.py ein.

Implementieren Sie die Ableitungen der unterschiedlichen Gütefunktionen, um möglichst weit von den Gelenkanschlägen weg bzw. möglichst nahe an der Komfortpose zu bleiben. Verwenden Sie diese dann in der Funktion für Automatic Supervisory Control. Fügen Sie die Gelenkwinkelbegrenzungen in der Konfigurationsdatei des Franka Robots ein. Sie finden die Begrenzungen in der Dokumentation des Franka Control Interfaces. Wir haben die erste Version, die auch *Panda* genannt wurde.

Sie können Ihre Implementierung auf Plausibilität überprüfen, indem Sie auch die Funktionen H_limit und H_comfort implementieren und sich die Gütemaße bei unterschiedlichen IKOptions ausgeben lassen.

2 Inverse Kinematik auf Positionsebene

Manchmal möchte man nicht von der aktuellen Gelenk- und Arbeitsraumposition starten und einer Trajektorie im Arbeitstraum folgen, sondern nur einzelne Positionen aus dem Arbeitsraum in den Gelenkraum übersetzten. Dafür benötigt man eine Inverse Kinematik auf Positionsebene. In der Vorlesung haben Sie bereits das Newton-Raphson Verfahren für die Kinematik auf Positionsebene kennen gelernt. Da es hierbei in der Nähe von Singularitäten zu Instabilität kommt, lernen Sie hier noch eine Erweiterung kennen, um den Startpunkt der im Arbeitsraum gegebenen Trajektorie in den Gelenkraum zu übersetzen.

Wir suchen einen Schritt $\Delta\boldsymbol{q}$, der uns von der aktuellen Gelenkkonfiguration näher an die Gelenkkonfiguration bringt, die die Arbeitsraumvorgabe erfüllt. Wir verwenden dafür den linearisierten kinematischen Zusammenhang $\Delta\boldsymbol{w} = \boldsymbol{J}\Delta\boldsymbol{q}$. Allerdings forden wir diesen linearisierten Zusammenhang nicht strikt ein, sondern bestrafen nur eine Abweichung davon und bestrafen gleichzeitig auch große Schritte:

$$ \min\left\{|\Delta\boldsymbol{w} - \boldsymbol{J}\Delta\boldsymbol{q}|^2 + \lambda^2\Delta\boldsymbol{q}^T\Delta\boldsymbol{q}\right\} $$

mit einem Dampfungskoeffizienten λ. Um den dafür besten nächsten Schritt $\Delta\boldsymbol{q}$ zu bestimmen, leiten wir nach $\Delta\boldsymbol{q}$ ab:

$$ \begin{aligned} 2\left(\Delta\boldsymbol{w} - \boldsymbol{J}\Delta\boldsymbol{q}\right)^T\left(-\boldsymbol{J}\right) + 2\lambda^2\Delta\boldsymbol{q}^T &= \boldsymbol{0} \\ -\boldsymbol{J}^T\left(\Delta\boldsymbol{w} - \boldsymbol{J}\Delta\boldsymbol{q}\right) + \lambda^2\Delta\boldsymbol{q} &= \boldsymbol{0} \\ \lambda^2\Delta\boldsymbol{q} &= \boldsymbol{J}^T\Delta\boldsymbol{w} - \boldsymbol{J}^T\boldsymbol{J}\Delta\boldsymbol{q} \\ \boldsymbol{J}^T\Delta\boldsymbol{w} &= \left(\boldsymbol{J}^T\boldsymbol{J} + \lambda^2\boldsymbol{I}\right)\Delta\boldsymbol{q} \\ \Delta\boldsymbol{q} &= \left(\boldsymbol{J}^T\boldsymbol{J} + \lambda^2\boldsymbol{I}\right)^{-1}\boldsymbol{J}^T\Delta\boldsymbol{w} \end{aligned} $$

Verwenden wir einen Dämpfungskoeffizierenten deutlich größer als 0, so kann der Ausdruck $\boldsymbol{J}^T\boldsymbol{J} + \lambda^2\boldsymbol{I}$ immer invertiert werden.

Mit einigem am Mathematik kann man zeigen, dass

$$ \left(\boldsymbol{J}^T\boldsymbol{J} + \lambda^2\boldsymbol{I}\right)^{-1}\boldsymbol{J}^T = \boldsymbol{J}^T\left(\boldsymbol{J}\boldsymbol{J}^T + \lambda^2\boldsymbol{I}\right)^{-1} $$

und somit können wir auch folgenden Ausdruck verwenden:

$$\Delta \boldsymbol{q} = J^T \left(\boldsymbol{J} \boldsymbol{J}^T + \lambda^2 \boldsymbol{I} \right)^{-1} \Delta \boldsymbol{w}$$

Hier müssen wir immer nur eine 6x6 Matrix invertieren, auch wenn wir mit Robotern mit mehr Freiheitsgraden arbeiten.

Ergänzen Sie die Funktion calculate_joint_angles in der Klasse IKSolverPositionDLS in der Datei rdplib / ik .py. Für λ ist bereits die Variable damping_factor in der Klasse angelegt. Führen Sie dann das Skript für Inverse Kinematik auf Positionsebene aus. Schauen Sie sich die Ergebnisse im Viewer an.